

03 Matrix Multiplication

3.4 The Fourth View of Matrix Multiplication

Linear Combinations of Row Vectors, the Transpose Relationship, and Heatmap Visualization



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

The linear-combination-of-rows view: for matrix multiplication AB , every row of the result is a weighted sum of B 's row vectors

2

For AB , each row of A acts as a set of weights applied to B 's rows, and the weighted sum produces the corresponding row of C

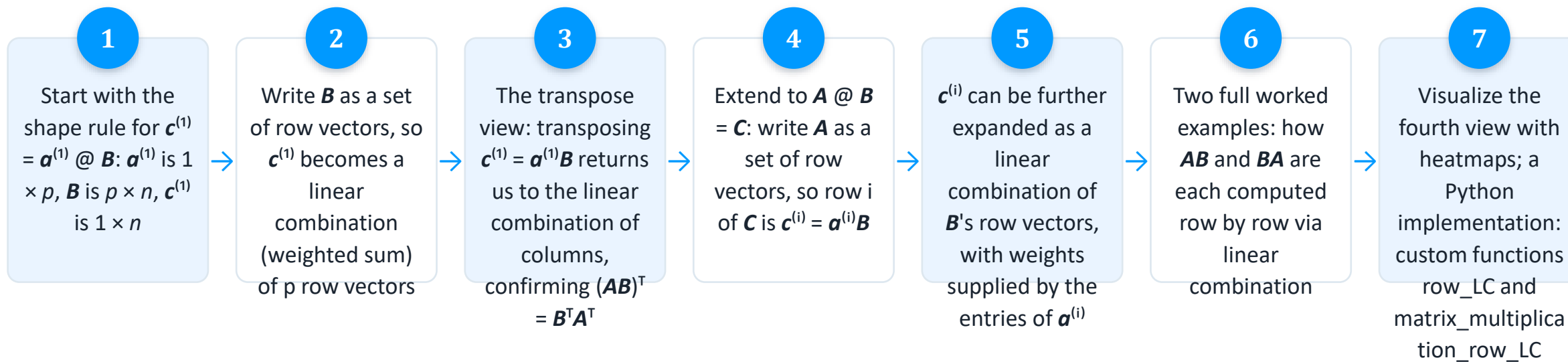
3

The row-vector view and the column-vector view of matrix multiplication convert into each other via the transpose

4

For AB , however many rows A has, AB has that many rows too

How This Section Unfolds



Seeing Matrix Multiplication as a Linear Combination of Rows

The fourth view of matrix multiplication writes the right-hand matrix as a set of row vectors — every row of the result is a linear combination of that set

- Since every column of $\mathbf{C}$ can be seen as a linear combination of $\mathbf{A}$'s columns, can $\mathbf{C}$ also be seen as some kind of linear combination of $\mathbf{B}$? The answer is yes
- The entries in each row of $\mathbf{A}$ serve as coefficients, weighting and summing $\mathbf{B}$'s row vectors to produce the corresponding row of result matrix $\mathbf{C}$

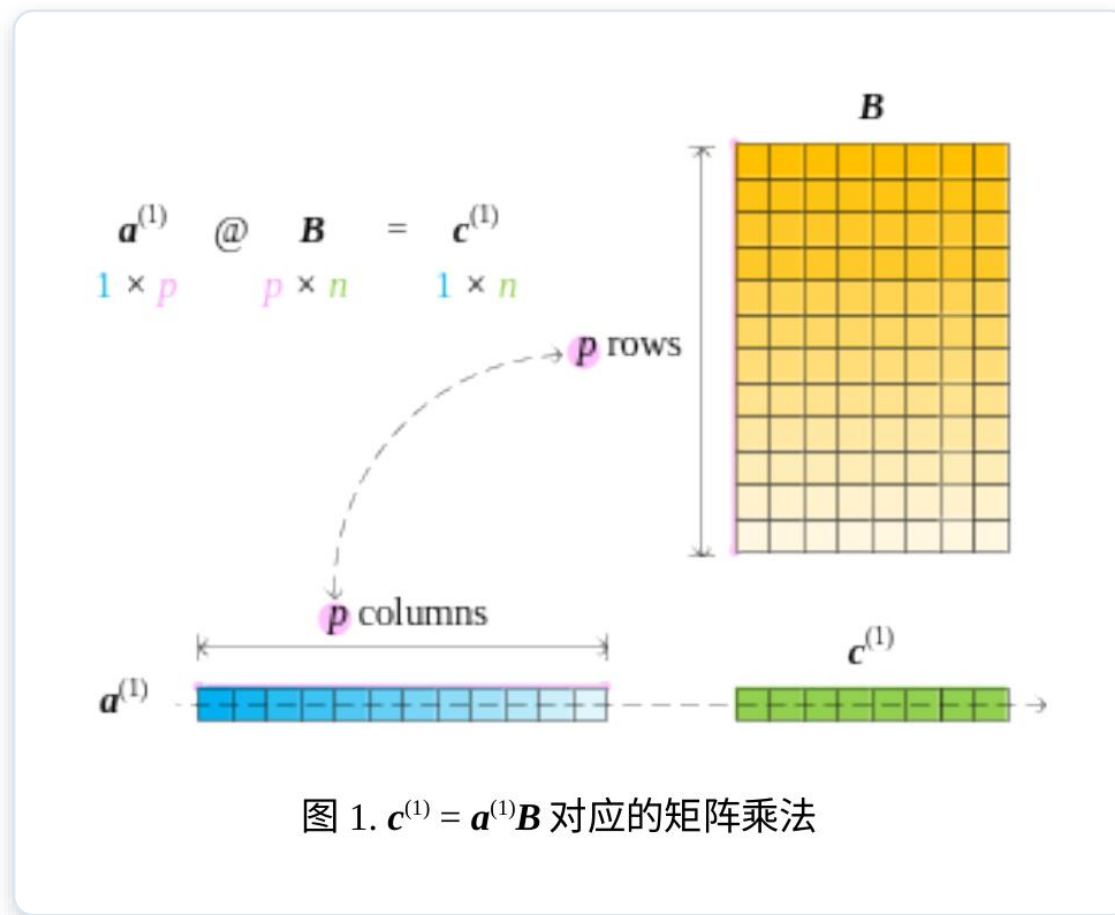
First, Look at the First Row of Matrix $\mathbf{C}$

$$\mathbf{c}^{(1)} = \mathbf{a}^{(1)} @ \mathbf{B}$$

- Take the first row of $\mathbf{C}$ as an example: $\mathbf{a}^{(1)}$ is $\mathbf{A}$'s first row, with shape $1 \times p$; $\mathbf{B}$ has shape $p \times n$; their product $\mathbf{c}^{(1)}$ has shape $1 \times n$
- This is the most basic case of a row vector multiplying a matrix, and also the starting point for understanding the fourth view

Figure 1 · The Matrix Multiplication Behind $\mathbf{c}^{(1)} = \mathbf{a}^{(1)}\mathbf{B}$

- $\mathbf{a}^{(1)}$ is a $1 \times p$ row vector, $\mathbf{B}$ is a $p \times n$ matrix; $\mathbf{a}^{(1)}$'s p columns line up with $\mathbf{B}$'s p rows, and the shared dimension p “cancels out”
- The result $\mathbf{c}^{(1)}$ has shape $1 \times n$, matching $\mathbf{B}$'s column count n



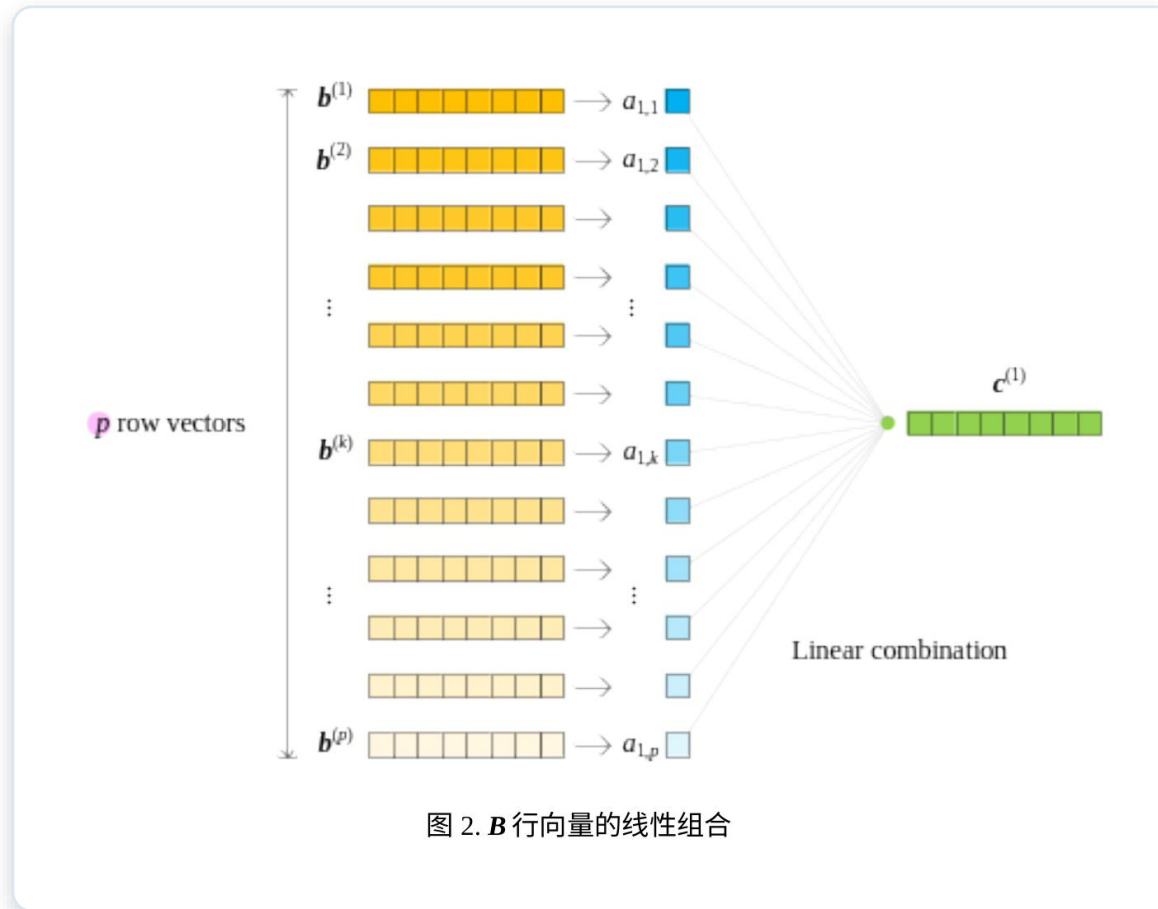
c(1) Can Be Written as a Linear Combination of **B**'s Rows

$$\mathbf{c}^{(1)} = \mathbf{a}^{(1)} @ \mathbf{B} = [a_{1,1} \ a_{1,2} \ \cdots \ a_{1,p}] [\mathbf{b}^{(1)}; \mathbf{b}^{(2)}; \vdots; \mathbf{b}^{(p)}] = a_{1,1}\mathbf{b}^{(1)} + a_{1,2}\mathbf{b}^{(2)} + \cdots + a_{1,p}\mathbf{b}^{(p)}$$

- Writing matrix **B** as a set of row vectors $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(p)}$, $\mathbf{c}^{(1)}$ is exactly a linear combination (weighted sum) of these p row vectors
- This is the fourth view of matrix multiplication, the linear-combination-of-rows view

Figure 2 · A Linear Combination of B's Row Vectors

- B 's p row vectors $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(p)}$ are each multiplied by the corresponding coefficient $a_{1,1}, a_{1,2}, \dots, a_{1,p}$ from $\mathbf{a}^{(1)}$
- Adding these p weighted row vectors together produces the result row vector $\mathbf{c}^{(1)}$ — that is exactly what “linear combination” means



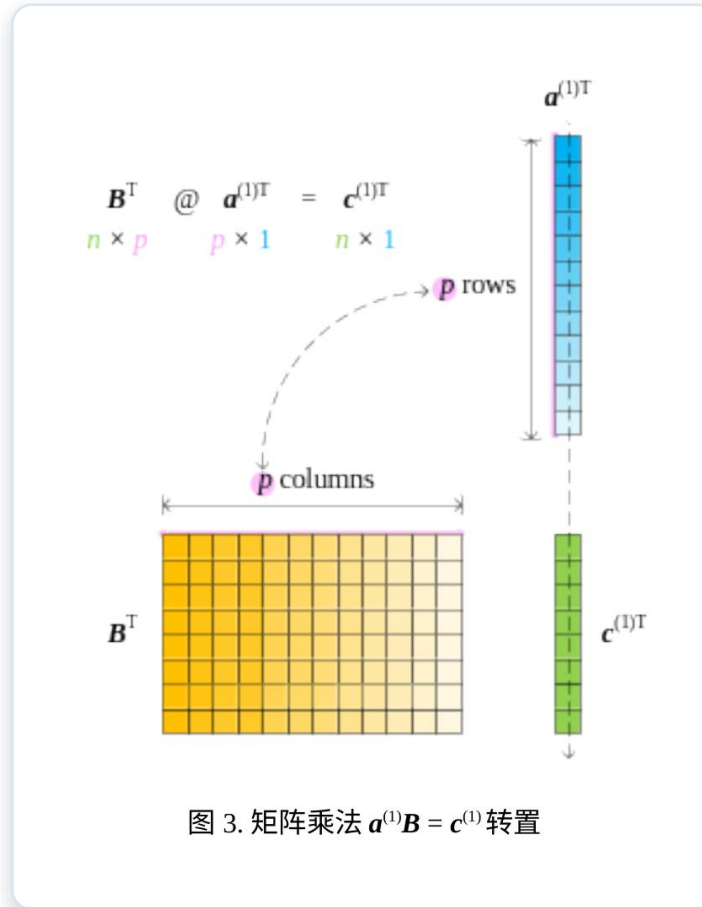
A Different Angle: Transposing $\mathbf{c}(1) = \mathbf{a}(1)\mathbf{B}$

$$\mathbf{c}^{(1)T} = \mathbf{B}^T @ \mathbf{a}^{(1)T}$$

- After transposing, we're back to a linear combination of columns — this also confirms, from another angle, that the transpose of $\mathbf{AB} = \mathbf{C}$ is $\mathbf{B}^T\mathbf{A}^T = \mathbf{C}^T$
- Note: $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$ — never forget to reverse the order

Figure 3 · Transposing the Matrix Multiplication $\mathbf{a}^{(1)}\mathbf{B} = \mathbf{c}^{(1)}$

- $\mathbf{B}^T$ is an $n \times p$ matrix, $\mathbf{a}^{(1)T}$ is a $p \times 1$ column vector; the transposed result $\mathbf{c}^{(1)T}$ is an $n \times 1$ column vector
- This is exactly the form of the previous section's linear-combination-of-columns view — the row view and column view are really two sides of the same coin



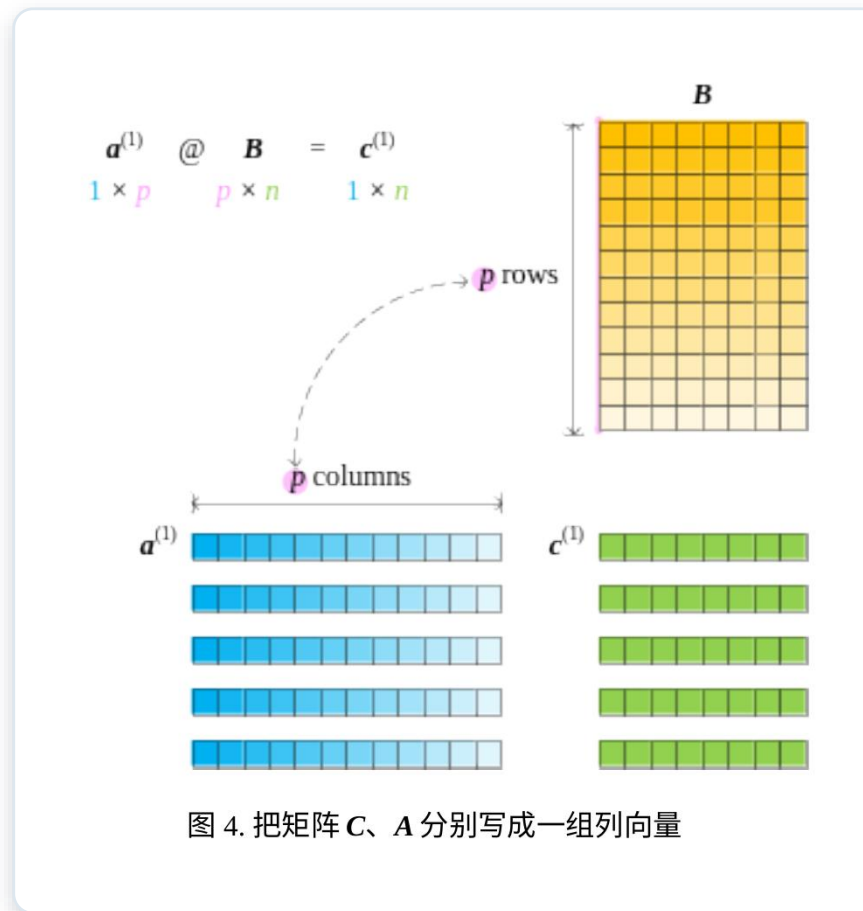
Now $\mathbf{A} @ \mathbf{B} = \mathbf{C}$

$$\mathbf{C} = [\mathbf{c}^{(1)}; \mathbf{c}^{(2)}; \vdots; \mathbf{c}^{(m)}] = [\mathbf{a}^{(1)}; \mathbf{a}^{(2)}; \vdots; \mathbf{a}^{(m)}] @ \mathbf{B} = [\mathbf{a}^{(1)}\mathbf{B}; \mathbf{a}^{(2)}\mathbf{B}; \vdots; \mathbf{a}^{(m)}\mathbf{B}]$$

- The linear-combination-of-rows view extends naturally to $\mathbf{C} = \mathbf{A} @ \mathbf{B}$: write $\mathbf{A}$ as a set of row vectors
- $\mathbf{A}$ has m rows, so $\mathbf{A} @ \mathbf{B}$ also has m rows — $\mathbf{A}$'s row count determines the product's row count

Figure 4 · Writing C and A as Sets of Row Vectors

- Each row $\mathbf{a}^{(i)}$ of $\mathbf{A}$ ($i = 1, 2, \dots, m$) corresponds to the row $\mathbf{c}^{(i)}$ in the same position of $\mathbf{C}$
- $\mathbf{a}^{(i)}$ is a $1 \times p$ row vector, $\mathbf{B}$ is a $p \times n$ matrix; the shared dimension p “cancels out,” leaving $\mathbf{c}^{(i)}$ with shape $1 \times n$



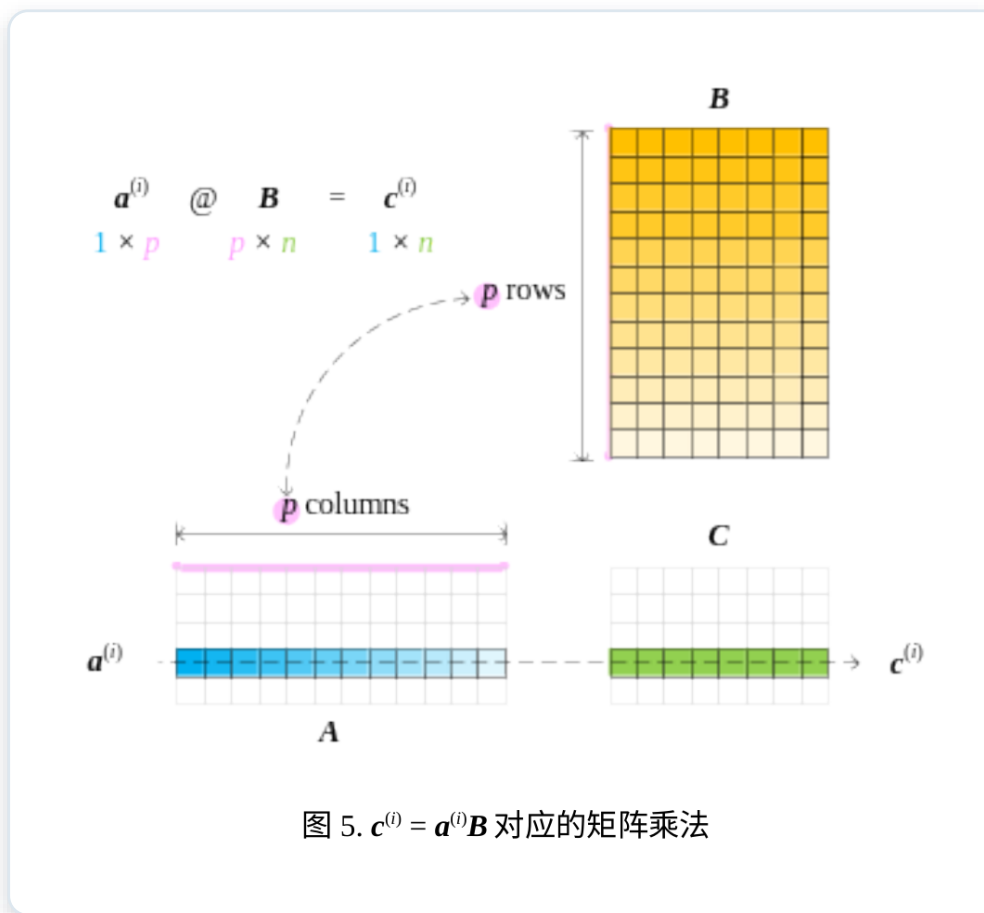
Any Row of $\mathbf{C}$ Can Be Computed Row by Row

$$\mathbf{c}^{(i)} = \mathbf{a}^{(i)} @ \mathbf{B}$$

- Row i of $\mathbf{C}$ is simply the product of $\mathbf{A}$'s row i , $\mathbf{a}^{(i)}$, with $\mathbf{B}$ — this breaks matrix multiplication into m separate “row vector times matrix” operations
- This is exactly the same case as Figure 1's $\mathbf{a}^{(1)}\mathbf{B} = \mathbf{c}^{(1)}$, just applied here to compute one row of $\mathbf{C}$ at a time

Figure 5 · The Matrix Multiplication Behind $c(i) = a(i)B$

- B stays fixed; taking row i of A , $a^{(i)}$, and multiplying by B gives exactly row i of C , $c^{(i)}$
- Repeating this for $i = 1$ through m computes every row of C , assembling the complete $C = A @ B$



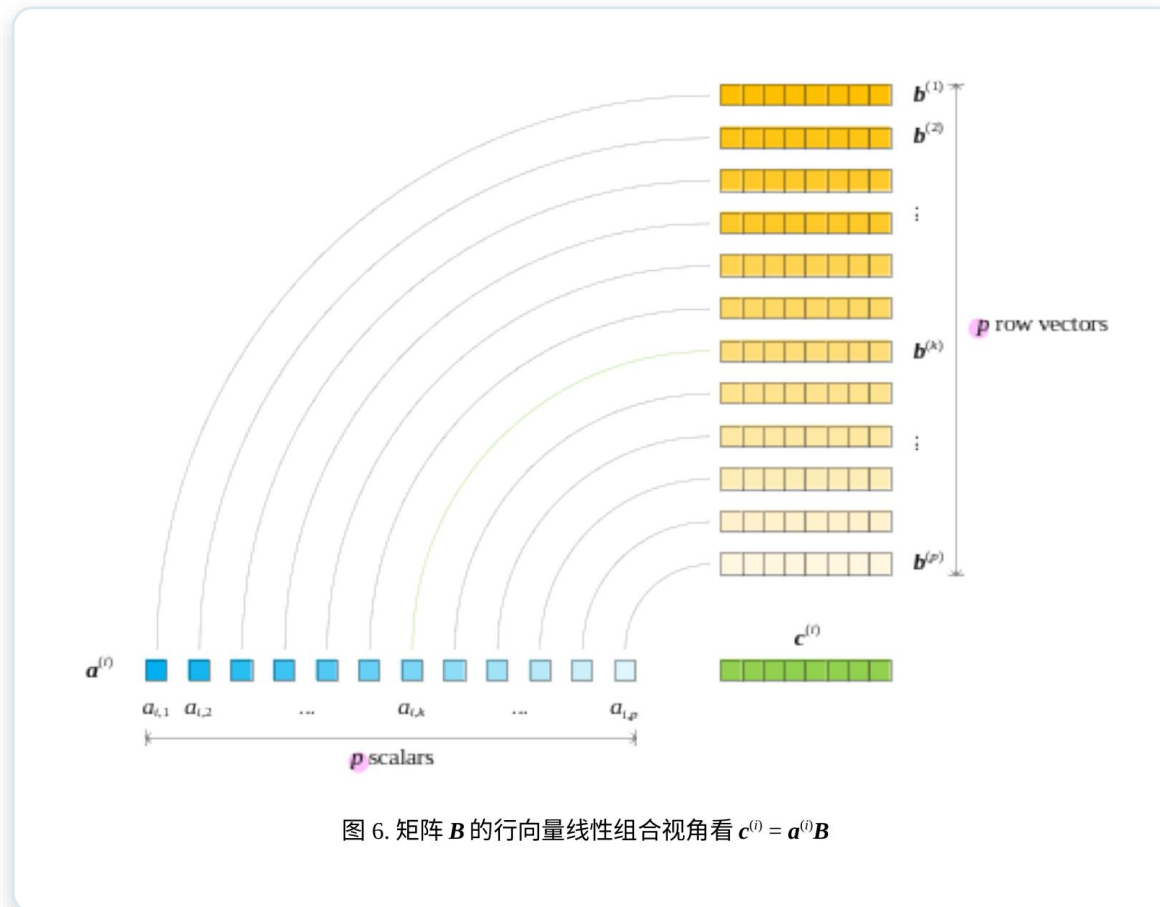
c(i) Can Be Further Written as a Linear Combination of B's Rows

$$\mathbf{c}^{(i)} = \mathbf{a}^{(i)} @ \mathbf{B} = [a_{i,1} \ a_{i,2} \ \cdots \ a_{i,p}] [\mathbf{b}^{(1)}; \mathbf{b}^{(2)}; \vdots; \mathbf{b}^{(p)}] = a_{i,1} \mathbf{b}^{(1)} + a_{i,2} \mathbf{b}^{(2)} + \cdots + a_{i,p} \mathbf{b}^{(p)}$$

- The combination coefficients come from row i of $\mathbf{A}$, $\mathbf{a}^{(i)}$ — its p scalar entries are exactly p weights
- Working backward: after computing $\mathbf{a}^{(1)}\mathbf{B}$, $\mathbf{a}^{(2)}\mathbf{B}$, ..., one by one, stacking them in order produces matrix $\mathbf{C}$

Figure 6 · The Linear-Combination View of $c(i) = a(i)B$

- $a^{(i)}$'s p entries $a_{i,1}, a_{i,2}, \dots, a_{i,p}$ are each the weight for one of B 's p row vectors $b^{(1)}, b^{(2)}, \dots, b^{(p)}$
- Summing the weighted rows gives $c^{(i)}$ — this figure unifies “row-by-row computation” and “linear combination of rows” into one picture



Back to an Earlier Example: Compute $c(1)$ and $c(2)$ via Linear Combination

$$\mathbf{c}^{(1)} = \mathbf{a}^{(1)} @ \mathbf{B} = [1,2,3] @ \mathbf{B} = 1 \times [1,4] + 2 \times [2,5] + 3 \times [3,6] = [14,32]; \mathbf{c}^{(2)} = [4,5,6] @ \mathbf{B} = 4 \times [1,4] + 5 \times [2,5] + 6 \times [3,6] = [32,77]$$

- $\mathbf{A} = [1,2,3; 4,5,6]$, $\mathbf{B} = [1,4; 2,5; 3,6]$; $\mathbf{B}$'s three rows $[1,4]$, $[2,5]$, $[3,6]$ are each weighted by the entries of $\mathbf{a}^{(1)}$ and $\mathbf{a}^{(2)}$
- The result matches what the earlier inner-product expansion and linear-combination-of-columns views computed exactly — different paths, same destination

Stacking $c(1)$ and $c(2)$ Gives Matrix C

$$C = A @ B = [c^{(1)}; c^{(2)}] = [14, 32; 32, 77]$$

- Once $c^{(1)}$ and $c^{(2)}$ are computed, stacking them in order assembles the complete C
- Whether you approach it via inner product, outer product, linear combination of columns, or linear combination of rows, the result is always the same

Figure 7 · Stacking $c(1)$ and $c(2)$ to Form Matrix C

- $c^{(1)} = [14, 32]$ comes from the linear combination of B 's rows with $a^{(1)} = [1, 2, 3]$; $c^{(2)} = [32, 77]$ comes from $a^{(2)} = [4, 5, 6]$
- Stacking $c^{(1)}$ and $c^{(2)}$ gives $C = [14, 32; 32, 77]$

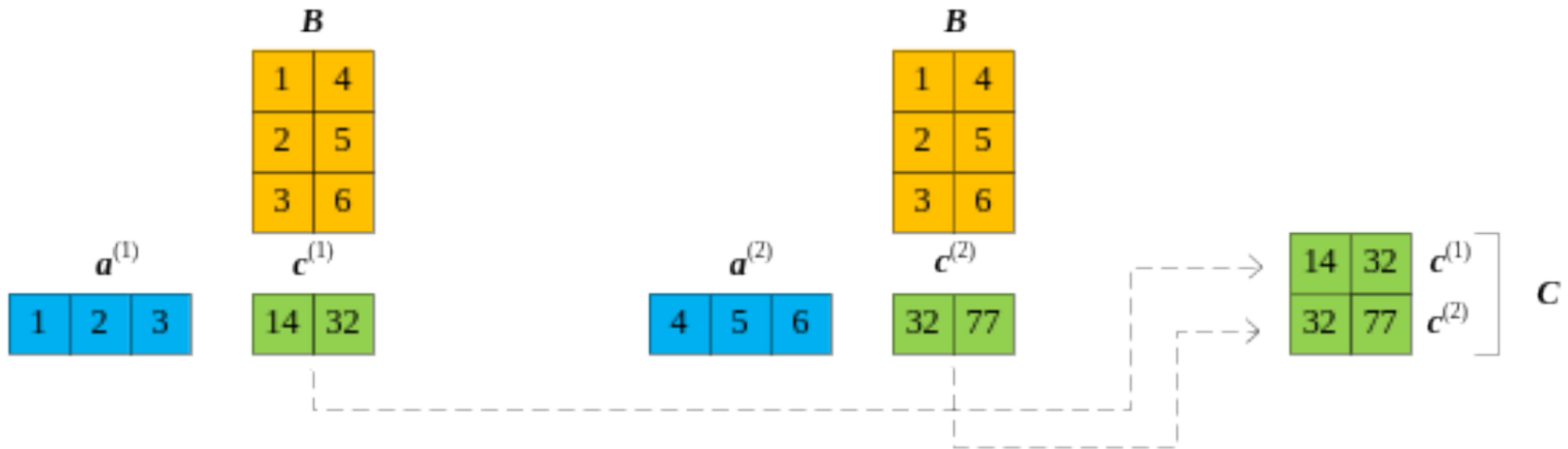


图 7. 把行向量 $c^{(1)}$ 、 $c^{(2)}$ 顺序摆放得到矩阵 C

SECOND EXAMPLE

The Linear-Combination-of-Rows View of the Second Example: $D = B @ A$

$$\mathbf{d}^{(1)} = \mathbf{b}^{(1)} @ \mathbf{A} = 1 \times [1, 2, 3] + 4 \times [4, 5, 6] = [17, 22, 27]; \mathbf{d}^{(2)} = 2 \times [1, 2, 3] + 5 \times [4, 5, 6] = [22, 29, 36]; \mathbf{d}^{(3)} = 3 \times [1, 2, 3] + 6 \times [4, 5, 6] = [27, 36, 45]$$

- Here the roles switch: the left-hand matrix is $\mathbf{B} = [1, 4; 2, 5; 3, 6]$, and the coefficients come from $\mathbf{B}$'s three rows $[1, 4]$, $[2, 5]$, $[3, 6]$, applied to $\mathbf{A}$'s two row vectors
- $\mathbf{B}$ has 3 rows, so $D = B @ A$ needs three linear combinations of rows

Figure 8 · Stacking $d^{(1)}$, $d^{(2)}$, and $d^{(3)}$ to Form Matrix D

- $d^{(1)}$, $d^{(2)}$, $d^{(3)}$ each get their weights from B 's 1st, 2nd, and 3rd rows, applied to A 's two row vectors
- Stacking the three rows in order assembles $D = [17, 22, 27; 22, 29, 36; 27, 36, 45]$, matching the results from earlier sections exactly

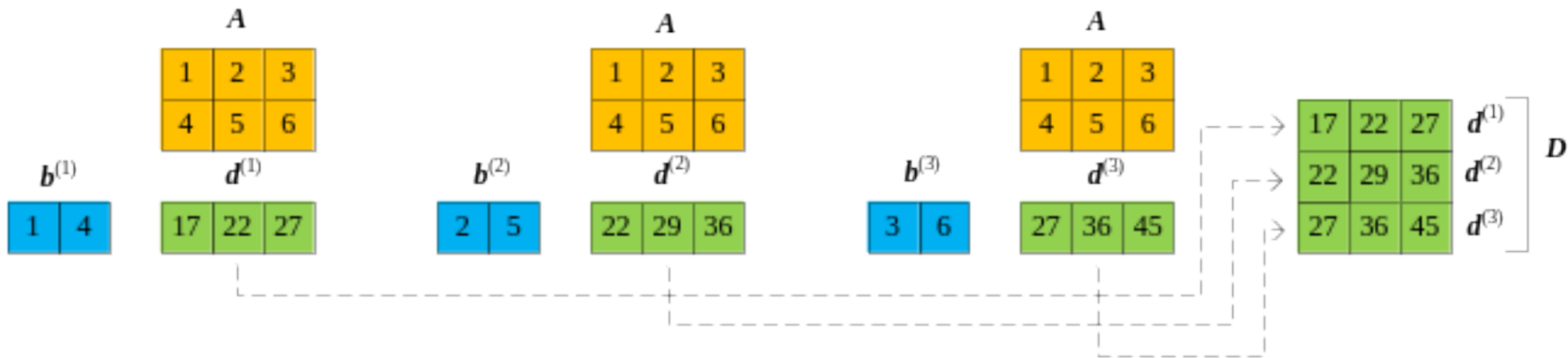


图 8. 把行向量 $d^{(1)}$ 、 $d^{(2)}$ 、 $d^{(3)}$ 顺序摆放得到矩阵 D

Using Heatmaps to Show the Fourth View of Matrix Multiplication

Color intensity represents the magnitude of each entry; heatmaps let us “watch” each row $a(i)B$ get computed and assembled one by one into $A @ B$

- Given a pair of matrices A , B , first look at the heatmap showing how $a^{(1)} @ B$ is computed
- Since A has 5 rows, there are 5 such products in total; stacking these 5 result rows assembles the complete $C = A @ B$

Figure 9 · Heatmap Showing $\mathbf{a}^{(1)} @ \mathbf{B}$

- $\mathbf{a}^{(1)}$ is $\mathbf{A}$'s first row (color represents magnitude), combined with $\mathbf{B}$'s rows via weighted sum to produce the $\mathbf{a}^{(1)} @ \mathbf{B}$ row heatmap
- This is the first step of row-by-row computation; the remaining four rows follow the exact same process

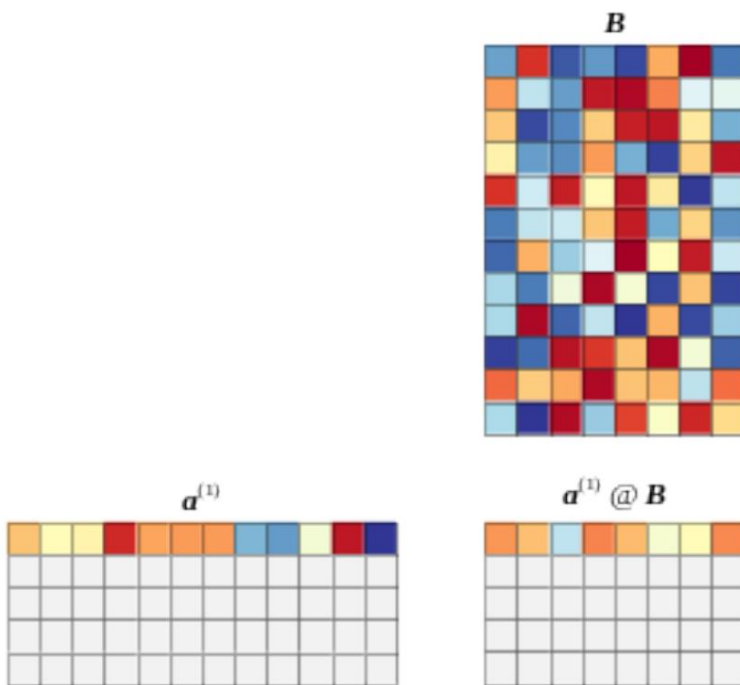


图 9. 热图展示 $\mathbf{a}^{(1)} @ \mathbf{B}$

Figure 10 · The 5 Products $\mathbf{a}^{(i)} @ \mathbf{B}$ Stacked to Form $\mathbf{C} = \mathbf{A} @ \mathbf{B}$

- $\mathbf{A}$ has 5 rows, so there are 5 products altogether, just like $\mathbf{a}^{(1)} @ \mathbf{B}$, corresponding to $\mathbf{c}^{(1)}$ through $\mathbf{c}^{(5)}$
- Stacking these 5 result rows in order assembles the complete $\mathbf{C} = \mathbf{A} @ \mathbf{B}$

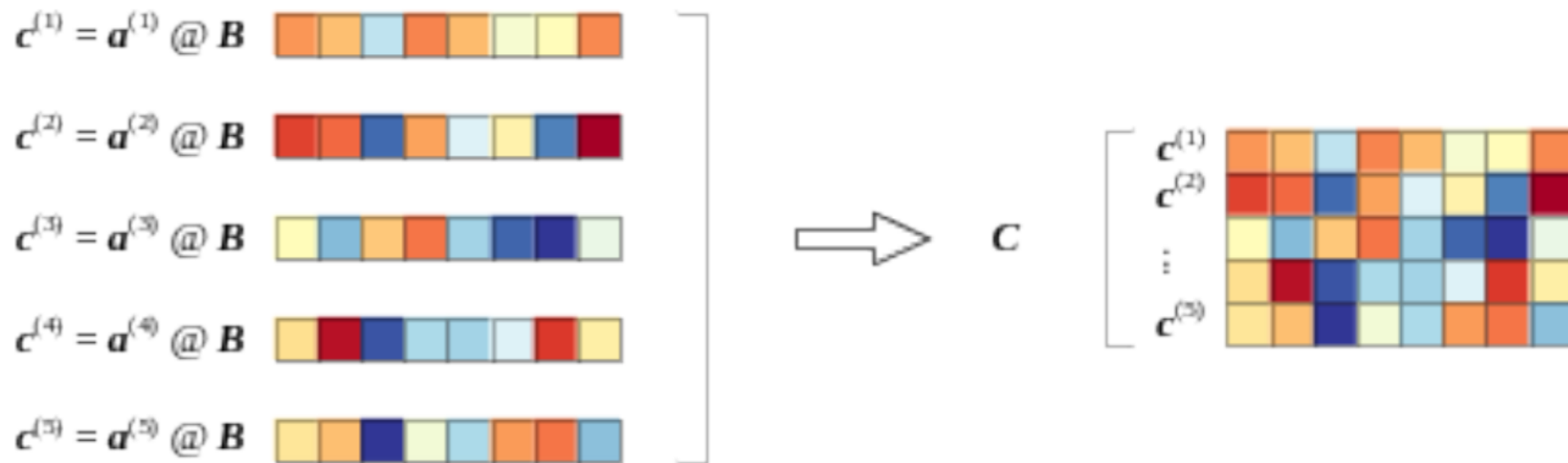


图 10. 5 个 $\mathbf{a}^{(i)} @ \mathbf{B}$ 上下排列得到矩阵 $\mathbf{C} = \mathbf{A} @ \mathbf{B}$

Computing Matrix Multiplication via Linear Combinations of Rows

First define $\text{row_LC}(B, \text{coefficients})$ to compute a linear combination of rows, then define $\text{matrix_multiplication_row_LC}(A, B)$, which calls it row by row to complete the whole multiplication

- row_LC uses a single for-loop over each row of B , multiplies it by the matching coefficient, and accumulates the result into a linear-combination vector
- $\text{matrix_multiplication_row_LC}$ uses a single for-loop over each row of A , passing that row as the coefficients to row_LC , filling result matrix C one row at a time — the code structure closely mirrors the previous section's linear-combination-of-columns view

Code 1 · A Custom Python Function for Matrix Multiplication

LA_03_04_03.ipynb

- `def row_LC(B, coefficients):` takes each row `B[k, :]`, multiplies by `coefficients[k]`, and accumulates — producing the linear-combination result
- `def matrix_multiplication_row_LC(A, B):` first checks whether `p_A` equals `p_B`, raising an error if not, then initializes result matrix `C` as an (m, n) matrix of zeros
- Looping `i = 0` to `m-1`, it takes row `i` of `A` as the coefficients (`coeffs = A[i, :]`) and calls `C[i, :] = row_LC(B, coeffs)` to fill in `C` one row at a time

```
## 初始化
import numpy as np

## 定义行向量线性组合函数
def row_LC(B, coefficients):
    p, n = B.shape
    row_LC_result = np.zeros(n)
    # 初始化行向量组合结果

    for k in range(p):
        row_LC_result += coefficients[k] * B[k,:]
        # 每行乘系数并相加
    return row_LC_result

## 定义矩阵乘法函数（行线性组合法）
def matrix_multiplication_row_LC(A, B):
    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C，形状 (m, n)，初始值设为 0
    C = np.zeros((m, n))

    for i in range(m):
        coeffs = A[i, :]
        # 取出 A 的第 i 列，作为线性组合的系数
        C[i, :] = row_LC(B, coeffs)
        # 计算 B 行向量的线性组合

    return C

## 矩阵乘法
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication_row_LC(A, B)
matrix_multiplication_row_LC(B, A)
```

Linear Combinations of Rows: The Fourth Key to Matrix Multiplication

Treat each row of A as a set of weights applied to B 's rows, combine and stack row by row — this is the fourth key to understanding the structure of matrix multiplication

- Inner-product expansion, outer-product expansion, linear combination of columns, and linear combination of rows are four different views of the same matrix multiplication AB , and they always agree
- The row view and column view convert into each other via the transpose: transpose a row-view conclusion and you get the corresponding column-view conclusion, and vice versa

Key Terms in This Section

Linear Combination of Row Vectors $a(1)B = a_{1,1}b(1) + a_{1,2}b(2) + \dots + a_{1,p}b(p)$

The Transpose Relationship $(AB)^T = B^T A^T$ — the row-vector view and column-vector view convert into each other via the transpose

Combination Coefficients The entries $a_{i,1}, a_{i,2}, \dots$ of $a(i)$ serve as the weights in the linear combination

Permutation Matrix (Left Multiplication) $P @ B$ reorders B 's rows

The `matrix_multiplication_row_LC` Function Calls `row_LC` row by row to complete the full matrix multiplication

The Fourth View of Matrix Multiplication Row i of C , $c(i)$, is a linear combination of B 's rows, weighted by row i of A

Row-by-Row Computation $c(i) = a(i)B$ — breaking matrix multiplication into m “row vector times matrix” steps

Identity Matrix Satisfies $I @ B = B$, using standard basis row vectors as coefficients

The `row_LC` Function A custom Python function computing a linear combination of rows given coefficients

Heatmap (Fourth View) A visualization showing each row $a(i)B$ computed and assembled into $A @ B$

GO FURTHER

Companion Code and Further Reading

Further Reading · LA_03_04_01.ipynb

Computes both worked examples in this section using the fourth view of matrix multiplication

Further Reading · LA_03_04_02.ipynb

Uses `seaborn.heatmap()` to visualize the fourth view of matrix multiplication

Further Reading · LA_03_04_03.ipynb

A custom Python function for matrix multiplication, implemented from the linear-combination-of-rows view

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Use the linear-combination-of-rows view to compute $a(1) @ B$ for the following pairs: ① $a(1)=[1,2]$, $B=[1,2;3,4]$; ② $a(1)=[1,2,3]$, $B=[1,2;3,4;5,6]$; ③ $a(1)=[1,2]$, $B=[1,2,3;4,5,6]$; ④ $a(1)=[1,2,3]$, $B=[1,2,3;4,5,6;7,8,9]$.

Q3 Combine the following two matrix multiplications into a single one: $[1,2;3,4]@[1;1]$ and $[1,2;3,4]@[-1;1]$.

Q5 Combine the following three matrix multiplications into a single one, and give the result: $[0,0,1]@[1,2,3;4,5,6;7,8,9]$, $[1,0,0]@[1,2,3;4,5,6;7,8,9]$, $[0,1,0]@[1,2,3;4,5,6;7,8,9]$.

Q7 Write a function `row_lc_matmul(a, B)` that computes $a @ B$ using the linear-combination-of-rows approach, and compare it against `np.dot(a, B)`.

Q9 Modify the `matrix_multiplication_row_LC` function from Code 1, constructing at least 3 shape-mismatched A, B pairs, and verify each correctly raises a `ValueError`.

Q2 Use the fourth view of matrix multiplication to expand AB for the following pairs: ① $A=[1,2;3,4]$, $B=[1,-1;1,1]$; ② $A=[1,2;3,4;5,6]$, $B=[1,-1;1,1]$; ③ $A=[1,2,3;4,5,6]$, $B=A^T$; ④ $A=[1,2,3;4,5,6;7,8,9]$, $B=A^T$.

Q4 Combine the following three matrix multiplications into a single one, and give the result: $[1,0,0]@[1,2,3;4,5,6;7,8,9]$, $[0,1,0]@[1,2,3;4,5,6;7,8,9]$, $[0,0,1]@[1,2,3;4,5,6;7,8,9]$.

Q6 Compare the results of Q4 and Q5, and explain their similarities and differences.

Q8 Use Python to verify the $a(1) @ B$ computations for the four pairs in Q1, printing each one's linear-combination expansion (e.g., “ $1 \times b(1) + 2 \times b(2)$ ”).

Questions Q10 – Q18

Q10 Use the time module to compare the runtime of `matrix_multiplication_row_LC` against NumPy's built-in `A @ B` on large matrices (e.g., 300×300).

Q12 Write a function `decompose_by_rows(A, B)` that decomposes `A @ B` into `m` row computations `a(i)B`, and use matplotlib subplots to show each row's computation step by step.

Q14 Construct a 3×3 identity matrix `I`, verify whether `I @ B` equals `B` itself, and explain why from the linear-combination-of-rows perspective.

Q16 Write a function `random_row_combination(B)` that randomly generates a coefficient vector `a`, computes `a @ B`, and plots `B`'s row vectors and the result `c` as arrows in a 2D plane (assuming `B` is a $p \times 2$ matrix).

Q18 Use Python to verify this section's second example: given `B=[1,4;2,5;3,6]`, `A=[1,2,3;4,5,6]`, compute `B @ A` using the linear-combination-of-rows view, and compare against earlier sections' results.

Q11 Compute the same pair of random matrices' product using custom functions for the third view (linear combination of columns) and fourth view (linear combination of rows), and verify the results agree exactly.

Q13 Use Python to verify the transpose relationship: for random `A` and `B`, compute `(A @ B).T` and `B.T @ A.T`, and verify they are exactly equal.

Q15 Construct a permutation matrix `P` (swap two rows of the identity matrix), compute `P @ B`, and observe how the result connects to the patterns in Q4 and Q5.

Q17 Use Python to verify this section's first example: given `A=[1,2,3;4,5,6]`, `B=[1,4;2,5;3,6]`, compute `A @ B` using the linear-combination-of-rows view, and compare against earlier sections' results.

Questions Q19 – Q26

Q19 Following Figures 9 and 10, draw heatmaps for a random pair of 5×4 and 4×6 matrices, showing the fourth view's row-by-row computation and assembly.

Q21 Explore: if some row $a(i)$ of matrix A is entirely zero, what is row i of $C = A @ B$? Verify your conjecture with Python.

Q23 Given a 3×3 matrix B , construct three different permutation matrices, left-multiply each by B , observe how the rows of the result get reordered, and summarize the pattern of what left-multiplying by a permutation matrix does.

Q25 Combining the third and fourth views of matrix multiplication, write a function `verify_transpose_view(A, B)` that checks whether computing $A @ B$ via the linear-combination-of-columns view and then transposing equals computing $(A @ B)$ directly via the linear-combination-of-rows view.

Q20 Write a function `check_zero_row(a, B)` that verifies whether $a @ B$ is always the zero vector when a is the zero vector, and explain why from the linear-combination perspective.

Q22 Write a function `compare_four_views(A, B)` that computes $A @ B$ using the first, second, third, and fourth views, tallying each implementation's line count and runtime, and summarizing the trade-offs of each view.

Q24 Write a function `matrix_from_row_basis(B, coeffs_list)` that takes matrix B and a list of coefficient vectors, and returns the new matrix formed by combining B 's rows according to each coefficient vector.

Q26 Combining what you learned in this section, write a small program that, given any matrices A and B , checks whether A 's column count equals B 's row count, and if so, prints the full linear-combination expansion of every row $c(i)$ (e.g., “ $c(1) = 1 \times b(1) + 2 \times b(2)$ ”).

10 Prompts to Explore Further (1/2)

1

Explain why matrix multiplication $a(1) @ B$ can be understood as a linear combination of B 's row vectors, and where the combination coefficients come from.

2

Explain how the third view (linear combination of columns) and fourth view (linear combination of rows) of matrix multiplication convert into each other via the transpose, illustrating with $(AB)^T = B^T A^T$.

3

Explain why row i of $A @ B$, $c(i)$, equals $a(i) @ B$, and how this connects to the other views.

4

Explain why the identity matrix I satisfies $I @ B = B$, from the perspective of linear combinations of rows.

5

Explain what left-multiplying by a permutation matrix does, and why $P @ B$ reorders B 's rows.

10 Prompts to Explore Further (2/2)

6

Explain why, in the heatmap visualization, computing each row $a(i)B$ and assembling them one by one demonstrates the fourth view of matrix multiplication.

7

Explain the complete relationship among all four views of matrix multiplication (inner-product expansion, outer-product expansion, linear combination of columns, linear combination of rows) — do they always produce the same result?

8

Explain the relationship between the linear-combination-of-rows view and the linear-algebra concept of “row space.”

9

Give an example of how the linear-combination-of-rows view is used in practice in machine learning or data science, such as weighted combinations of samples.

10

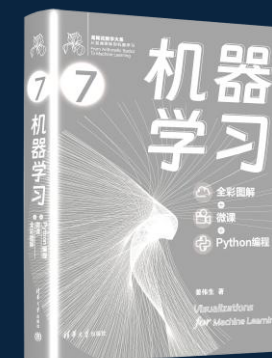
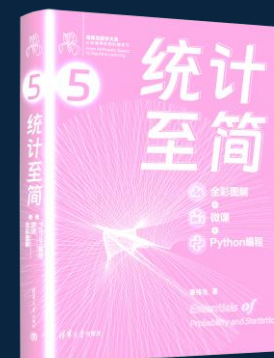
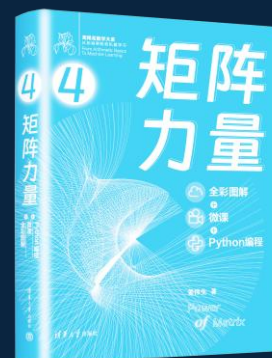
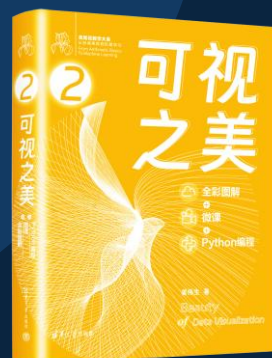
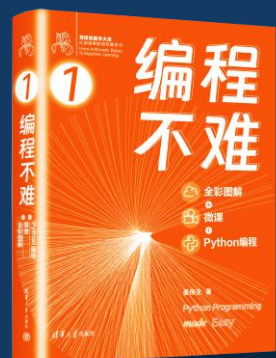
Explain why understanding the linear-combination-of-rows view of matrix multiplication is essential for later topics such as matrix rank, row space, and systems of linear equations.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

